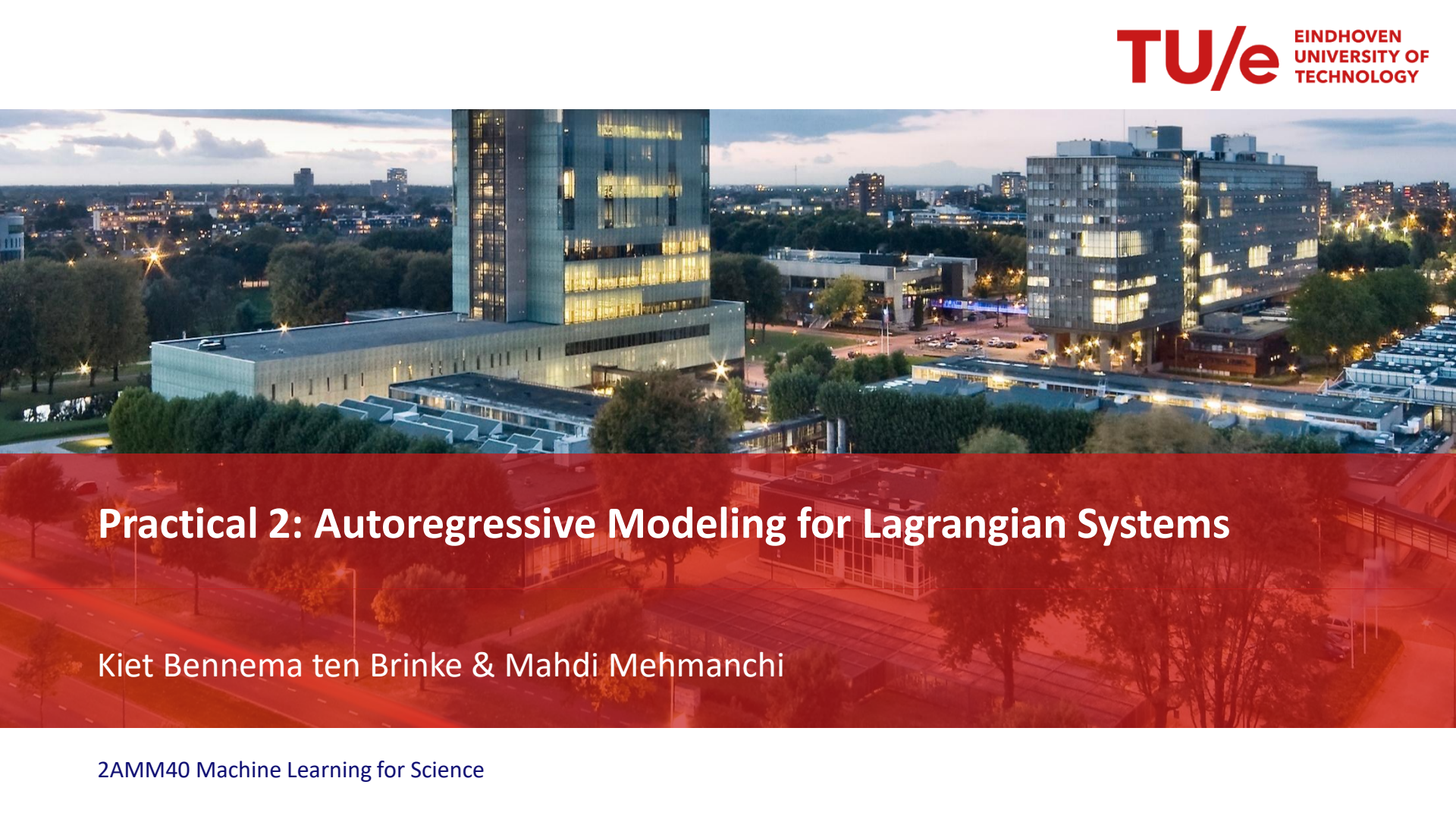


An aerial night photograph of the TU/e campus in Eindhoven, showing several modern glass-walled buildings illuminated from within. The image is partially covered by a semi-transparent red rectangle that serves as a background for the title and authors.

Practical 2: Autoregressive Modeling for Lagrangian Systems

Kiet Bennema ten Brinke & Mahdi Mehmanchi

Neural Message Passing Code

$$x_i^{(k)} = \gamma^{(k)}(x_i^{(k-1)}, \bigcup_{j \in \mathcal{N}(i)} \phi^{(k)}(x_i^{(k-1)}, x_j^{(k-1)}, e_{j,i}))$$

- $\phi^{(k)}$ - Edge transfer function at iteration k
- $\gamma^{(k)}$ - Node update function at iteration k
- $\bigcup$ - Message aggregation function
- $\mathcal{N}(i)$ - Neighbour nodes to node i
- $x_i^{(k)}$ - Representation of node i at iteration k
- $e_{j,i}$ - Representation of edge between nodes i and j

```
i, j = edge_index
messages = self.message_mlp(T.cat([x[i], x[j], edge_attr], dim=-1))

aggr_messages = T.zeros(N, messages.size(-1), device=x.device)
aggr_messages.index_reduce_(dim=0, index=j, source=messages, reduce="mean")

x = T.cat(tensors=[x, aggr_messages], dim=-1)
x = self.node_mlp(x)
```



E(n)-equivariant GNN Code

Edge message:

$$m_{ij} = \phi(h_i^{(k)}, h_j^{(k)}, \|x_i^{(k)} - x_j^{(k)}\|^2, e_{ij})$$

Node feature update:

$$h_i^{(k)} = \gamma \left(h_i^{(k-1)}, \sum_{j \neq i} m_{ij} \right)$$

Velocity update:

$$v_i^{(k+1)} = \psi(h_i^{(k)})v_i^{(0)} + C \sum_{j \neq i} (x_i^{(k)} - x_j^{(k)}) \zeta(m_{ij})$$

Coordinate update:

$$x_i^{(k+1)} = x_i^{(k)} + \dot{x}_i^{(k+1)} \Delta t$$

```
# Edge messages: m_ij = phi_e(h_i, h_j, ||x_i - x_j||^2, a_ij)
m = self.edge_mlp(
    T.cat(tensors=[h[src], h[dst], dist2, edge_attr], dim=-1)
)

# Aggregate messages: m_i = sum_j m_ij
agg = h.new_zeros(h.shape[0], m.shape[-1])
agg.index_add_(0, dst, m)

# Node update: h_i^{l+1} = phi_h(h_i, m_i)
h_new = self.node_mlp(
    T.cat(tensors=[h, agg], dim=-1)
)

# Velocity update: v_i^{l+1} = phi_v(h_i) * v_i^{init} + C * sum
velocity_scale = self.velocity_mlp(h)

dv = (x[src] - x[dst]) * self.coord_mlp(m)
interaction = v_init.new_zeros(v_init.shape)
interaction.index_add_(0, dst, dv)

v_new = velocity_scale * v_init + self.C * interaction

# Coordinate update: x_i^{l+1} = x_i + v_i^{l+1}
x_new = x + v_new
```